

Exercícios de autorrevisão

- 24.1** Liste cinco exemplos comuns de exceções.
- 24.2** Dê alguns motivos pelos quais as técnicas de tratamento de exceção não devem ser usadas no controle convencional do programa.
- 24.3** Por que as exceções são apropriadas para lidar com erros produzidos pelas funções de biblioteca?
- 24.4** O que é uma ‘perda de recurso’?
- 24.5** Se nenhuma exceção é disparada em um bloco `try`, para onde o controle prossegue depois que o bloco `try` completa sua execução?
- 24.6** O que acontece se uma exceção for disparada fora de um bloco `try`?
- 24.7** Explique a principal vantagem e a principal desvantagem de usar `catch(...)`.
- 24.8** O que acontece se nenhum tratador de `catch` corresponder ao tipo de um objeto disparado?
- 24.9** O que acontece se vários tratadores combinarem com o tipo do objeto disparado?
- 24.10** Por que você especificaria um tipo de classe-base como o tipo de um tratador de `catch`, e depois dispararia objetos dos tipos da classe derivada?
- 24.11** Suponha que um tratador de `catch` com uma correspondência exata com um tipo de objeto de exceção esteja disponível. Sob quais circunstâncias um tratador diferente seria executado para os objetos de exceção desse tipo?
- 24.12** O disparo de uma exceção deve causar o término do programa?
- 24.13** O que acontece quando um tratador de `catch` dispara uma exceção?
- 24.14** O que a instrução `throw`; faz?
- 24.15** Como você restringe os tipos de exceção que uma função pode disparar?
- 24.16** O que acontece se uma função disparar uma exceção de um tipo não permitido pela especificação de exceção da função?
- 24.17** O que acontece com os objetos automáticos que foram construídos em um bloco `try` quando esse bloco dispara uma exceção?

Respostas dos exercícios de autorrevisão

- 24.1** Memória insuficiente para satisfazer uma solicitação `new`, subscrito de array fora dos limites, overflow aritmético, divisão por zero, parâmetros de função inválidos.
- 24.2** (a) O tratamento de exceção foi projetado para lidar com situações que ocorrem com pouca frequência, que normalmente resultam no término do programa, de modo que os escritores de compilador não precisam implementar o tratamento de exceção para que eles funcionem de modo ideal. (b) O fluxo de controle com estruturas de controle convencionais geralmente é mais claro e mais eficiente que aquele com exceções. (c) Pode haver problemas porque a pilha está aberta quando ocorre uma exceção, e os recursos alocados antes da exceção podem não ser liberados. (d) As exceções ‘adicionais’ fazem com que seja mais difícil para você tratar de um grande número de casos de exceção.
- 24.3** É improvável que uma função de biblioteca realize um processamento de erro que atenda às necessidades exclusivas de todos os usuários.
- 24.4** Um programa que termina bruscamente poderia deixar um recurso em um estado no qual outros programas não poderiam adquiri-lo, ou o próprio programa poderia não conseguir readquirir um recurso ‘perdido’.
- 24.5** Os tratadores de exceção (nos tratadores de `catch`) para esse bloco `try` são pulados, e o programa continua a ser executado após o último tratador de `catch`.
- 24.6** Uma exceção disparada fora de um bloco `try` faz com que uma chamada termine (com `terminate`).
- 24.7** A forma `catch(...)` captura qualquer tipo de exceção disparada em um bloco `try`. Uma vantagem é que todas as exceções possíveis serão capturadas. Uma desvantagem é que o `catch` não tem parâmetro, de modo que não pode referenciar informações no objeto disparado e não pode saber a causa da exceção.
- 24.8** Isso faz com que a busca por uma correspondência continue no próximo bloco `try` delimitador, se houver um. Com a continuação desse processo, por fim será determinado que não existe tratador no programa que corresponda ao tipo do objeto disparado; nesse caso, `terminate` é chamada e, automaticamente, chama `abort`. Uma função `terminate` alternativa pode ser fornecida como um argumento para `set_terminate`.
- 24.9** O primeiro tratador de exceção correspondente após o bloco `try` é executado.

- 24.10** Porque esta é uma boa maneira de capturar (com `catch`) os tipos de exceções relacionados.
- 24.11** Um tratador da classe base capturaria objetos de todos os tipos da classe derivada.
- 24.12** Não, mas isso termina o bloco em que a exceção foi disparada.
- 24.13** A exceção será processada por um tratador de `catch` (se houver um) associado ao bloco `try` (se houver um) delimitando o tratador de `catch` que causou a exceção.
- 24.14** Ela indica a exceção se aparecer em um tratador de `catch`; caso contrário, a função `unexpected` é chamada.
- 24.15** Fornecendo uma especificação de exceção listando os tipos de exceção que a função pode disparar.
- 24.16** A função `unexpected` é chamada.
- 24.17** O bloco `try` expira, fazendo com que os destrutores sejam chamados em cada um desses objetos.

Exercícios

- 24.18** Liste as diversas condições excepcionais que ocorreram ao longo deste capítulo. Cite o máximo de condições excepcionais adicionais que você puder. Para cada uma dessas exceções, descreva resumidamente como um programa normalmente trataria da exceção, usando as técnicas de tratamento de exceção discutidas aqui. Algumas exceções típicas são divisão por zero, overflow aritmético, subscripto de array fora do limite, falta de espaço no armazenamento livre etc.
- 24.19** Sob quais circunstâncias você não forneceria um nome de parâmetro ao definir o tipo do objeto que será apanhado pelo tratador?
- 24.20** Um programa contém a instrução `throw`;
Onde você normalmente esperaria encontrar tal instrução? E se essa instrução aparecesse em uma parte diferente do programa?
- 24.21** Compare o tratamento de exceção com os vários outros esquemas de processamento de erros discutidos no texto.
- 24.22** Por que as exceções não devem ser usadas como forma alternativa de controle de programa?
- 24.23** Descreva uma técnica para tratar de exceções relacionadas.
- 24.24** *Disparando exceções de um `catch`.* Suponha que um programa dispare uma exceção, e o tratador de exceção apropriado comece a ser executado. Agora suponha que o próprio tratador de exceção dispare a mesma exceção. Isso cria uma recursão infinita? Escreva um programa para justificar a sua resposta.
- 24.25** *Captura de exceções da classe derivada.* Use a herança para criar várias classes derivadas de `runtime_error`. Depois, mostre que um tratador de `catch` que especifica a classe-base pode capturar (`catch`) exceções da classe derivada.
- 24.26** *Disparando o resultado de uma expressão condicional.* Dispare o resultado de uma expressão condicional que retorna um `double`, ou um `int`. Forneça um tratador `int catch` e um tratador `double catch`. Mostre que somente o tratador `double catch` é executado, independentemente de `int` ou `double` ser retornado.
- 24.27** *Destrutores de variável local.* Escreva um programa que demonstre que todos os destrutores de objetos construídos em um bloco são chamados antes de uma exceção ser disparada desse bloco.
- 24.28** *Destrutores de objeto-membro.* Escreva um programa que demonstre que os destrutores de objeto-membro são chamados somente naqueles objetos membros que foram construídos antes que ocorresse uma exceção.
- 24.29** *Captura de todas as exceções.* Escreva um programa que demonstre vários tipos de exceção sendo capturados com o tratador de exceção `catch(...)`.
- 24.30** *Ordem dos tratadores de exceção.* Escreva um programa que demonstre que a ordem dos tratadores de exceção é importante. O primeiro tratador correspondente é aquele que é executado. Tente compilar e executar seu programa de duas maneiras diferentes, para mostrar que dois tratadores diferentes são executados com dois efeitos diferentes.
- 24.31** *Construtores que disparam exceções.* Escreva um programa que mostre um construtor passando informações sobre falhas do construtor a um tratador de exceção após um bloco `try`.
- 24.32** *Indicação de exceções.* Escreva um programa que demonstre a indicação de uma exceção.
- 24.33** *Exceções não capturadas.* Escreva um programa que demonstre que uma função com seu próprio bloco `try` não precisa capturar cada erro possível gerado dentro do `try`. Algumas exceções podem passar e ser tratadas em escopos mais externos.
- 24.34** *Desempilhamento.* Escreva um programa que dispare (`throw`) uma exceção a partir de uma função profundamente aninhada e ainda faça com que o tratador de `catch` após o bloco `try`, que delimita a cadeia de chamadas, capture a exceção.